

Programowanie obiektowe i inżynieria oprogramowania

Lab AG-5:

- rozwiązanie zadania: klasa populacji,
- dynamiczne tworzenie obiektów.

Akademia Górniczo-Hutnicza im. Stanisława Staszica w Krakowie
AGH University of Science and Technology

Prowadzący: dr inż. Jakub Bryła
2025

Klasa populacji

Kod zadania

```
7   using namespace std;
8
9
10  int main()
11  {
12      srand(time(0));
13
14      TPopulation pop(10);
15
16      cout << "Zaraz po utworzeniu obiektu klasy TPopulaiton";
17      pop.info();
18      cout << "\n\n";
19
20      cout << "Po wykonanych obliczeniach";
21      pop.calculate();
22      pop.info();
23      cout << "\n\n";
24
25      cout << "Najlepszy osobnik:";
26      pop.get_best_candidate().info();
27
28      return 0;
29  }
```

Konsola debugowania programu Microsoft Visual Studio

Zaraz po utworzeniu obiektu klasy TPopulaiton

```
===== POPULATION #0 =====
== candidate#0: 0
== candidate#1: 0
== candidate#2: 0
== candidate#3: 0
== candidate#4: 0
== candidate#5: 0
== candidate#6: 0
== candidate#7: 0
== candidate#8: 0
== candidate#9: 0
=====
```

Po wykonanych obliczeniach

```
===== POPULATION #0 =====
== candidate#0: 38
== candidate#1: 25
== candidate#2: 69
== candidate#3: 107
== candidate#4: 6
== candidate#5: 36
== candidate#6: 64
== candidate#7: 18
== candidate#8: 9
== candidate#9: 10
=====
```

Najlepszy osobnik:

```
=====
==
== gens count: 2
== "x1" value: 10
== "x2" value: 7
==
== rate: 107
=====
```



```
TPopulation.h*  main.cpp  TParam.h  TCandidate.h
AG2024  (Globalny zasięg)
1  #pragma once
2
3  #include <vector>
4  #include "TCandidate.h"
5
6  class TPopulation
7  {
8  static unsigned int  _id;
9  unsigned int  candiated_count;
10 std::vector<TCandidate>  candidates;
11 double  best_val = 0;
12
13 public:
14     TPopulation(unsigned int cands_count = 10);
15     void  calculate();
16     TCandidate  get_best_candidate();
17
18     void  info();
19 };
```

Linia 8: **statyczny** parametr składowy.

```
TPopulation.cpp*  TParam.cpp
AG2024 (Globalny zasięg)
1  #include <iostream>
2  #include <algorithm> // max
3
4  #include "TPopulation.h"
5
6  using namespace std;
7
8  unsigned int TPopulation::_id = 0;
9
10 TPopulation::TPopulation(unsigned int cand_count)
11 {
12     _id++;
13     cand_count = cand_count;
14
15     for (int i = 0; i < cand_count; i++) candidates.push_back({});
16     // for (int i = 0; i < cand_count; i++) candidates.push_back(TCandidate{});
17 }
```

Linia 8: nadanie wartości początkowej **statycznemu** parametrowi składowemu.

Konstruktor

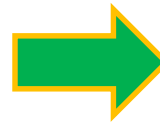
Linia 12: zasygnalizowanie stworzenia nowej populacji – zwiększenie wartości statycznego parametru składowego.

```
44 void TPopulation::info()  
45 {  
46     cout << "\n\n";  
47     cout << "==== POPULATION #" << _id << " =====\n";  
48  
49     for (int i = 0; i < candidated_count; i++)  
50     {  
51         cout << "== candidate#" << i << ": " << candidates[i].get_mark() << "\n";  
52     }  
53  
54     cout << "=====\n\n";  
55 }
```

Linia 47: wyświetlenie informacji o numerze utworzonej populacji.

Dynamiczne tworzenie obiektów

```
main.cpp  TKlasa.h
pomoc_populacja (Globalny zasięg)
1  #include <iostream>
2  #include <vector>
3  #include "TKlasa.h"
4
5  using namespace std;
6
7  int main()
8  {
9      TKlasa* wsk_obiekt1 = new TKlasa(-3);
10     TKlasa* wsk_obiekt2 = new TKlasa(0);
11     TKlasa* wsk_obiekt3 = new TKlasa(5);
12
13     wsk_obiekt1->info();
14     (*wsk_obiekt2).info();
15     wsk_obiekt3->info();
16
17     delete wsk_obiekt1;
18     delete wsk_obiekt2;
19     delete wsk_obiekt3;
20
21     return 0;
22 }
```



```
Konsola debugowania programu Microsoft Visual Studio
Obiekt klasy TKlasa o numerze: 1/3
Wartosci: -3, -2, -1, 0, 1,
Obiekt klasy TKlasa o numerze: 2/3
Wartosci: 0, 1, 2, 3, 4,
Obiekt klasy TKlasa o numerze: 3/3
Wartosci: 5, 6, 7, 8, 9,
```

Linie 17-19: dynamiczne tworzenie obiektów pozwala na podejmowanie decyzji, w którym momencie mają one zostać usunięte. Pozwala to na zarządzania pamięcią dostępną dla programu – szczególnie w przypadku bardzo dużych obiektów.

Dynamiczne tworzenie obiektów

Instrukcja *delete* pozwala usunąć utworzony dynamicznie obiekt. C++ zajmie się zwolnieniem pamięci zajmowanej przez obiekt. Dodatkowe operacje podczas usuwania obiektu można wywołać definiując **destruktor**.

```
TKlasa.h  dynamiczneTw...Obiektow.cpp
dynamiczneTworzenieObiektow  TKlasa
1  #pragma once
2  class TKlasa
3  {
4      static unsigned int count;
5      static unsigned int vector_size;
6      int init_val;
7      unsigned int _id;
8
9  public:
10     TKlasa(int init_val);
11     ~TKlasa();
12
13     void set_vector_size(unsigned int size);
14     void set_init_val(int init_val);
15     void info();
16 };
17
```

```
TKlasa.cpp
dynamiczneTworzenieObiektow  TKlasa  info()
1  #include "TKlasa.h"
2  #include <iostream>
3
4  using namespace std;
5
6  unsigned int TKlasa::count = 0;
7  unsigned int TKlasa::vector_size = 5;
8
9  TKlasa::TKlasa(int init_val)
10 {
11     TKlasa::count += 1;
12     _id = TKlasa::count;
13
14     this->init_val = init_val;
15 }
16
17 TKlasa::~TKlasa()
18 {
19     cout << "Usunieto obiekt klasy TKlasa o numerze: " << _id << " || ";
20
21     TKlasa::count -= 1;
22     cout << "Pozostaly " << TKlasa::count << " obiekty klasy TKlasa\n\n";
23 }
24
25 void TKlasa::set_vector_size(unsigned int size)
```

Dynamiczne tworzenie obiektów



```
TKlasa.h dynamiczneT...Obiektow.cpp
dynamiczneTworzenieObiektow (Globalny zasięg)
1 // dynamiczneTworzenieObiektow.cpp : Ten plik zawiera funkcję „main”. W nim
2 //
3
4 #include <iostream>
5 #include "TKlasa.h"
6 using namespace std;
7
8 int main()
9 {
10     TKlasa* wsk_obiekt1 = new TKlasa{ -3 };
11     TKlasa* wsk_obiekt2 = new TKlasa{ 0 };
12     TKlasa* wsk_obiekt3 = new TKlasa{ 5 };
13     TKlasa obiekt4{ -3 };
14     TKlasa obiekt5{ 2 };
15
16     wsk_obiekt1->info();
17     (*wsk_obiekt2).info();
18     wsk_obiekt3->info();
19     obiekt4.info();
20     obiekt5.info();
21
22     cout << "\n=====\\n\\n\\n";
23
24     delete wsk_obiekt1;
25     delete wsk_obiekt2;
26     delete wsk_obiekt3;
27
28     return 0;
29 }
```

Destruktor wywoływany jest dla każdego usuwanego obiektu, również usuwanego automatycznie przez C++ (obiekty o `_id = 4` i `_id = 5`).

W przypadku automatycznie usuwanych obiektów *stos* czyszczony jest od ostatniego utworzonego elementu. Najpierw usuwany jest obiekt o `_id = 5`, później o `_id = 4`.

Konsola debugowania programu Microsoft Visual Studio

Obiekt klasy TKlasa o numerze: 1/5
Wartosci: -3, -2, -1, 0, 1,

Obiekt klasy TKlasa o numerze: 2/5
Wartosci: 0, 1, 2, 3, 4,

Obiekt klasy TKlasa o numerze: 3/5
Wartosci: 5, 6, 7, 8, 9,

Obiekt klasy TKlasa o numerze: 4/5
Wartosci: -3, -2, -1, 0, 1,

Obiekt klasy TKlasa o numerze: 5/5
Wartosci: 2, 3, 4, 5, 6,

=====

Usunieto obiekt klasy TKlasa o numerze: 1 || Pozostaly 4 obiekty klasy TKlasa

Usunieto obiekt klasy TKlasa o numerze: 2 || Pozostaly 3 obiekty klasy TKlasa

Usunieto obiekt klasy TKlasa o numerze: 3 || Pozostaly 2 obiekty klasy TKlasa

Usunieto obiekt klasy TKlasa o numerze: 5 || Pozostaly 1 obiekty klasy TKlasa

Usunieto obiekt klasy TKlasa o numerze: 4 || Pozostaly 0 obiekty klasy TKlasa

```
24 delete wsk_obiekt1;  
25 wsk_obiekt1 = nullptr;  
26 delete wsk_obiekt2;  
27 wsk_obiekt2 = nullptr;  
28 delete wsk_obiekt3;  
29 wsk_obiekt3 = nullptr;
```

Linie 25, 27, 29: dobrym nawykiem jest nadanie wskaźnikowi wartości *nullptr* po usunięciu obiektu na który wskazywał.

Wskazywanie na miejsce, pod którym nie ma już obiektu generuje niebezpieczeństwo nadpisania nowej zawartości pamięci, co może powodować błędne działanie programu.